Manual for Stridulant: a Python Toolkit for Detecting Hidden Ecological Processes Through Sound

van Steijn, T.L.H., Rodriguez-Martinez, S. Klaminder, J.

October 27, 2026

Contact

PLEASE NOTE: this version of the manual was submitted alongside the manuscript '*Detecting Hidden Ecological Processes Through Sound: The Stridulant package*' and will not be updated. We recommend visiting the *Github* page to ensure you are accessing the latest version of the manual.

For questions, suggestions, or contributions regarding *Stridulant*, please contact:

- **T.L.H. van Steijn** — tamara.van.steijn@slu.se
- **S. Rodriguez-Martinez** — saro@norceresearch.no

Contents

1 Installation **2**
- 1.1 Download . . . 2
- 1.2 Install . . . 2
- 1.3 Updating the program . . . 3

2 Basic functionality **4**
- 2.1 Snippets . . . 5
- 2.2 Acoustic events and features . . . 5
- 2.3 Spectrograms . . . 8
- 2.4 Normalization . . . 11
- 2.5 Filters . . . 11
- 2.6 Augmentation methods . . . 11
  - 2.6.1 Gaussian noise . . . 11
  - 2.6.2 Time stretch . . . 12
  - 2.6.3 Pitch shift . . . 12

2.6.4 Time shift 12
2.6.5 Clipping distortion 13
2.6.6 Background noise 13
2.6.7 Time mask 13

3 Preparing snippet sets **14**
3.1 Processing annotation tables 14
3.2 Processing audio files 15
3.3 Extracting features 15

4 Feature-based scans **18**
4.1 Included functions 18
4.2 User-defined feature-based scans 20

5 Convolutional neural network **21**

6 Pipeline **22**

1 Installation

1.1 Download

Please download and unpack the anonymized version of *Stridulant* from Zenodo in your folder of choice.

1.2 Install

The easiest way is to install Stridulant with pip install. With a python interpreter (Bash, Spyder, Studio Visual Code or whichever you are using) navigate to the Stridulant folder.

```
# Navigate to the Stridulant folder, for example: #
cd stridulant

# Install package #
pip install −e .

# To uninstall, run #
pip uninstall −y stridulant
```

Now that the package is installed you can import it like any other module in Python.

```
import stridulant as st
```

Stridulant is now ready to use. Note that many functions in this manual have the same arguments. Their definitions are consistent throughout the manual and are thus not repeated in each section.

1.3 Updating the program

Anytime an update is available, go to git bash and execute:

```
# Navigate to the Stridulant folder #
cd path_to_folder/stridulant

# Update #
git pull origin master
```

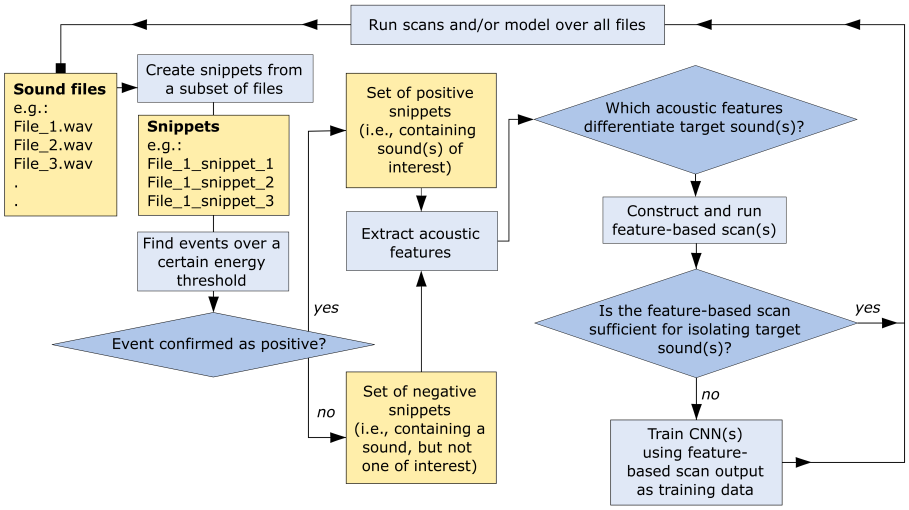


Figure 1: An example of a basic workflow of *Stridulant* using the snippet logic. Yellow boxes indicate sets of audio files, light blue boxes indicated actions done by *Stridulant* and blue diamond shaped boxes indicate decisions to be made by the user.

2 Basic functionality

All code in this manual can be found in the Stridulant installation folder under Tutorials

Stridulant's main task is to ease the preparation of sounds for custom feature-based scans or for AI training (Fig. 1). To do this, audio files are broken down into snippets. These snippets are fragments of a sound file that can then be easily handled and saved. This section describes all the functions that can be applied on single snippets and can be incorporated into the pipeline.

2.1 Snippets

Single snippets are created as follows:

```
# Load an audio file #
audio, sr  = st.load_audio(path_to_audio)

# Create a snippet #
Snippet = st.create_snippet(audio, sr, start_time, duration)

# Play snippet #
Snippet.play()

# Save snippet #
Snippet.save(source_name, output_dir, verbose, metadata)
```

path_to_audio	Path of audio file. Supported files are WAV and FLAC.
source_name	Base name for the saved file (typically the source file name).
output_dir	Directory where the snippet will be saved.
start_time	Start time of the snippet within the audio file in seconds.
duration	Length of snippet in seconds.
metadata	If True, appends snippets name with the snippet start time. Default is True
verbose	If True, prints a message when the snippet is saved. Default is True

2.2 Acoustic events and features

From snippets, events can be detected after which acoustic features can be extracted and saved (Table 1).

```
# Find events within a snippet #
events = Snippet.find_events(min_event_duration, env_smooth, threshold_percentile)

# Extract features from event #
features = Snippet.extract_features(events[0], pulse_dist, env_smooth,
    sustain_threshold_percentile, pulse_threshold_percentile, n_mfcc, preemp_coef, **
    mfcc_kwargs)

# Save features #
st.save_features(features, output_path)
```

min_event_duration	Minimum duration of events in seconds (default: 0.2).
env_smooth	Smooth factor for the Hilbert envelope. It is a number in milliseconds that smooths the peaks on that time range. Often numbers around 10 give a good trade-off between the smoothness of the line and the retention of features. Smaller numbers will give higher details, bigger numbers smoother lines (Figure 2). Make sure that int(env_smooth/1000*sr)>0.
threshold_percentile	Percentile value for energy threshold (0-100) (default: 25) (Figure 3).
pulse_dist	Minimum time between detectable pulses in milliseconds. Default is 20 ms.
sustain_threshold_percentile	The percentile of the smoothed envelope used to count pulses. Default is 30.
pulse_threshold_percentile	The percentile of the smoothed envelope used to calculate sustain, i.e. the ratio of the event containing audio. Default is 70.
n_mfcc	The number of Mel-frequency cepstral coefficient dimensions to extract. For each, statistics will be calculated and returned (Table 2). If 0 (default), no coefficients will be calculated.
preemp_coef	Pre-emphasis coefficient. Typical values of "coef" are between 0 and 1. Default is 0.97. See librosa's documentation for more information.
****mfcc_kwargs**	Additional arguments passed to librosa.feature.mfcc. See librosa's documentation for a complete list of supported parameters.
output_path	Path where the features will be saved ending in .csv.

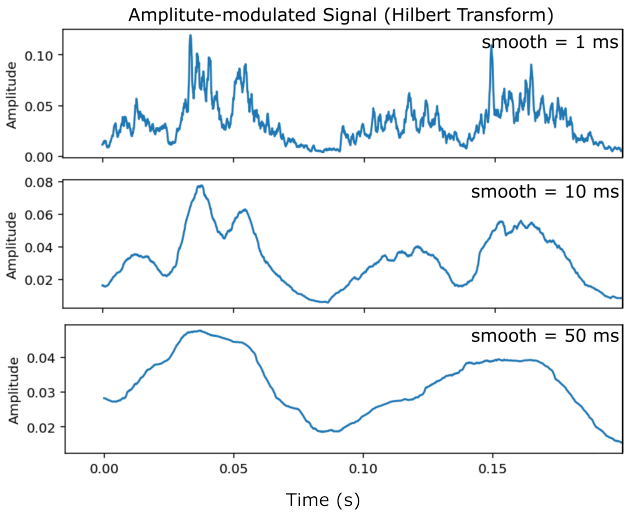


Figure 2: Visualization of the effect of different smoothing factors on Hilbert-transformed spectrograms. Lower smoothing factors preserve greater temporal and spectral detail but may fragment a single acoustic event into multiple detections. Higher smoothing factors reduce fine-scale variation and can facilitate the detection of longer, more continuous events.

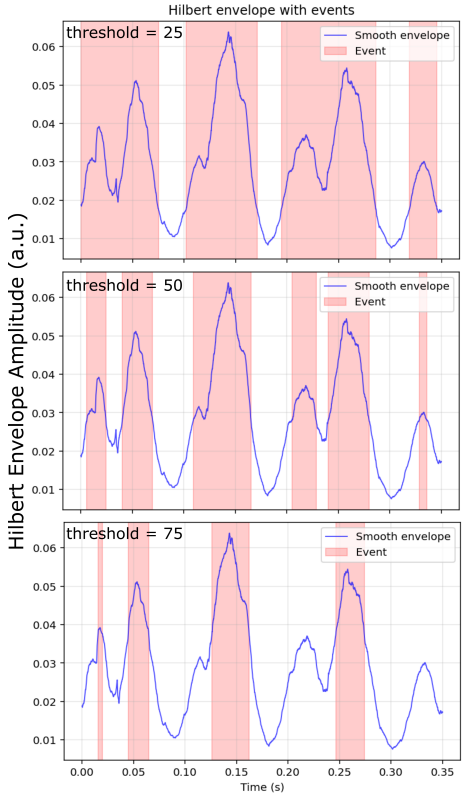


Figure 3: Visualization of the effect of different threshold percentiles on event selection. The threshold percentile is used to calculate an absolute amplitude threshold from the smoothed envelope. Events are defined as continuous periods during which the smoothed envelope remains above this threshold for at least a minimum duration. For example, in the first panel, the threshold is set to the 25th percentile. The audio snippet consists of 16,800 samples, with amplitude values ranging from 0.0075 to 0.0638. At the 25th percentile, 25% of the samples have an amplitude below 0.0177. Thus, 0.0177 is used as the absolute amplitude threshold. All consecutive regions where the smoothed envelope exceeds this threshold are initially grouped into events. These events are then filtered according to the specified minimum event duration. In this example, the minimum event duration is set to 0.001 seconds, ensuring that all detected events are retained.

2.3 Spectrograms

Snippets can be used to create spectrograms as follows:

```
# Create spectrogram object #
```

```
spectrogram = Snippet.spectrogram (spec_type, n_fft, n_mels, hop_length, window,
    env_smooth)

# Plot spectrogram #
spectrogram.plot(color)
spectrogram.plot_events(events)

# Save spectrogram object #
spectrogram.save_img(name, output_dir, with_labels, events)
```

spec_type	Type of the spectrogram ('mel', 'fft' or 'hilbert'). Default is 'mel' (Fig. 4)
hop_length	Overlap between windows. Bigger numbers (up to the window size) will make the process faster at the expense of temporal resolution, but also will make windows more independent from each other. Big numbers are best for power analysis, smaller numbers are better very short sounds and high temporal resolution (visualization and perhaps CNNs)
window	Shape of the window. Windows are related to leakage, which is a 'spill' of energy from a frequency bin to the adjacent one. The default is 'boxcar', which has a lot of leakage, but also the best frequency resolution. Check Librosa documentation for more options.
n_mels	Use when spec_type is set to 'mel'. Higher numbers increase the resolution in the frequency domain. It is best to use powers of 2
n_fft	Use when spec_type is set to 'fft'. This is the size of the window analyzed. Bigger numbers will increase resolution in frequency and smaller numbers increase resolution in time. It is a lot more efficient if the number is a power of 2.
color	Colormap for the spectrogram. Use plt.colormaps() to learn about your options.
with_labels	Add axis labels and legend to the spectrogram. Default is True.
events	List of events created by find_events(). If provided, shows where in the snippet acoustic events are found. Only applicable to Hillbert spectrograms (Fig. 5).

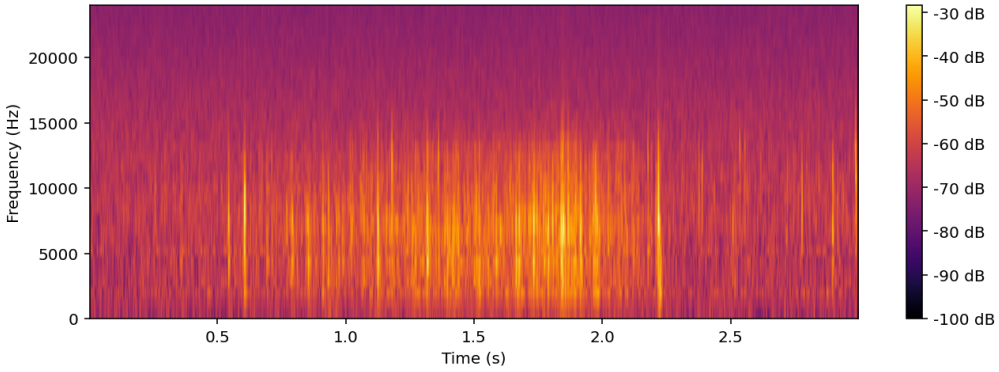


Figure 4: Mel spectrogram.

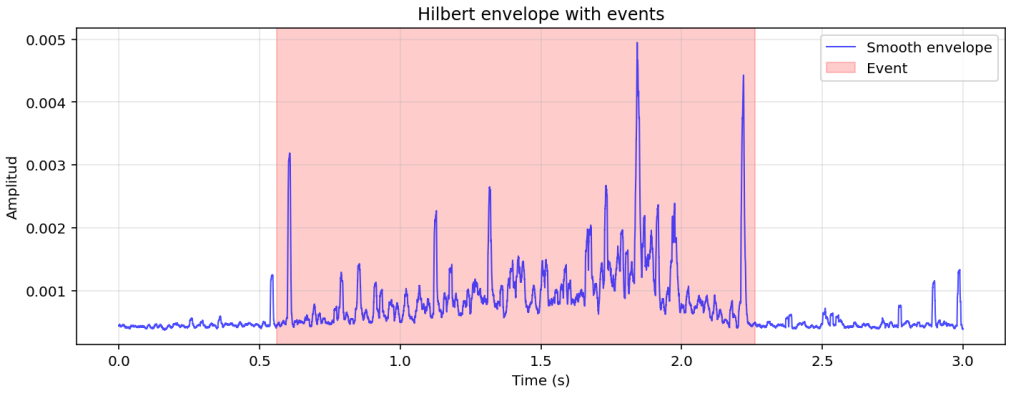


Figure 5: Hilbert spectrogram with event detection.

2.4 Normalization

There are three normalization functions that operate at different levels, i.e. they pick the maximum to normalize to from a different set: 1) within one snippet, 2) within one audio file and 3) across several audio files based on the global maximum. All levels of normalization can be included in the feature-based scan pipeline.

```
# Normalize within a single snippet #
Snippet_1_normalize = Snippet_1.normalize()

# Normalize within one audio file #
audio_normalize = st.normalize_audio(audio, target_max)

# Normalize across files #
normalize_global(input_dir, output_dir, target_max)
```

audio	The input audio signal.
target_max	Maximum percentage for the peak value (default 75%)
input_dir	Directory of input files to normalize globally.
output_dir	Where the normalized files will be saved.

2.5 Filters

High- and low-pass filters can be added, for example to filter out rain or to focus on the ultrasonic range.

```
Snippet_filtered = Snippet

# Highpass filter #
Snippet_filtered.audio = st.highpass_filter(Snippet.audio, sr, cutoff, order)

# Lowpass filter #
Snippet_filtered.audio = st.lowpass_filter(Snippet.audio, sr, cutoff, order)
```

audio	Input audio signal.
sr	Sampling rate of the audio
cutoff	Cutoff frequency in Hz.
order	Filter order. Default is 8 (48 dB/octave roll-off).

2.6 Augmentation methods

Stridulant includes wrappers of several functions from the Python package *Audiomentations*. These augmentations can be used to increase the diversity of snippet qualities, which is especially useful when there is only few training data.

2.6.1 Gaussian noise

Gaussian noise can be added to the audio signal to simulate real-world disturbances.

```
Snippet.add_gaussian_noise(
min_amplitude,
max_amplitude
)
```

min_amplitude Minimum amplitude for the Gaussian noise. Default is 0.001.

max_amplitude Maximum amplitude for the Gaussian noise. Default is 0.015.

2.6.2 Time stretch

Time stretching changes the speed of the audio signal without changing its pitch. The rate is selected randomly within the specified range.

```
Snippet.time_stretch(
min_rate,
max_rate
)
```

min_rate Minimum time-stretching rate. A value below 1 slows down the audio. Default is 0.8.

max_rate Maximum time-stretching rate. A value above 1 speeds up the audio. Default is 1.25.

2.6.3 Pitch shift

Pitch shifting changes the perceived pitch of the audio by a randomly selected number of semitones within the specified range.

```
Snippet.pitch_shift(
min_semitones,
max_semitones
)
```

min_semitones Minimum number of semitones to shift the pitch. Negative values shift the pitch down. Default is -4.

max_semitones Maximum number of semitones to shift the pitch. Positive values shift the pitch up. Default is 4.

2.6.4 Time shift

The audio signal can be shifted in time by a randomly selected fraction of its length. Negative values shift the audio backward, while positive values shift it forward.

```
Snippet.shift(
min_fraction,
max_fraction
)
```

min_fraction Minimum fraction of the signal to shift backward. Default is -0.5, corresponding to up to 50% of the signal length.

max_fraction Maximum fraction of the signal to shift forward. Default is 0.5, corresponding to up to 50% of the signal length.

2.6.5 Clipping distortion

Clipping distortion limits the amplitude of the audio signal, causing values outside the selected range to be clipped. The clipping percentage is selected randomly within the specified range.

```
Snippet.clipping_distortion(
min_percent,
max_percent
)
```

min_percent Minimum clipping percentage. Default is 10.

max_percent Maximum clipping percentage. Default is 30.

2.6.6 Background noise

Background noise can be added by mixing the audio signal with a background noise recording. The influence of the background noise is selected randomly within the specified range. The background data should ideally consist of noise-only snippets with the same length as the input audio.

```
Snippet.add_background_noise(
background_data,
min_background_influence,
max_background_influence
)
```

background_data Background noise used for mixing with the audio. Ideally, a folder of noise-only snippets with the same length as the input audio.

min_background_influence Minimum influence of the background noise. Default is 0.1.

max_background_influence Maximum influence of the background noise. Default is 0.3.

2.6.7 Time mask

A time mask randomly removes a portion of the audio signal by reducing the amplitude of a selected segment to zero. The length of the masked segment is determined as a proportion of the total audio duration.

```
Snippet.time_mask(
min_band_part,
max_band_part
)
```

min_band_part	Minimum proportion of the audio to mask. Default is 0.1 (10% of the audio).
max_band_part	Maximum proportion of the audio to mask. Default is 0.2 (20% of the audio).

3 Preparing snippet sets

Snippet sets can be created from annotation tables made in, for example, Raven or Audacity. Alternatively, a subset of audio files can be turned into snippets, on which event detection can be run. See the *Pepare snippet sets* tutorial.

3.1 Processing annotation tables

This function allows users to convert annotation tables into snippet sets. The default values match annotation tables made in Raven.

```
# Convert any annotation table into a snippet set #
process_table(input_table, output_dir, delim, audio, make_snippets,
      start_time_column, end_time_column, snippet_duration, ids, make_spectrograms,
       color, with_labels, verbose, metadata, **spectrogram_kwargs)
```

input_table	Path to a flat table in formats such as csv and txt.
output_dir	Where to save snippets and spectrogram folders.
delim	Symbol used to separate columns. Default is ‘,’.
audio	Path to audio.
make_snippets	If True, audio snippets will be saved.
start_time_column	The name of the column containing event start times. Default is `Begin Time (s)`.
end_time_column	The name of the column containing event end times. Default is `End Time (s)`.
snippet_duration	If specified, snippets will all be the same length. The start and end time columns will be used to find the middle between the beginning and end time, on which the snippet will be centered.
ids	Identifying columns, such as treatment, channel, selection, etc. Default is `["Selection"]`.
make_spectrograms	If True, spectrograms will be saved.
color	Colormap used for spectrograms. Default is `inferno`.
with_labels	If True, labels will be included in spectrograms. Default is True.
metadata	If True, metadata will be added to the snippet names. Default is True.
spectrogram_kwargs	Additional keyword arguments passed to the spectrogram generation function.

3.2 Processing audio files

Alternatively, event detection can be used to scan for acoustic events without an annotation table. To do this, an audio file will be clipped into snippets. Each snippet will then be scanned for events. Snippets with events will be copied into a separate folder. Events can then be manually sorted into target and non-target sound folders.

```
# Cut an audio file into snippets #
st.process_audio_file(audio_path, output_folder, snippet_duration, make_
    spectrograms, with_labels, color, overlap, **spectrogram_kwargs)

# Subset snippets containing events #
st.event_extractor(snippet_folder, event_snippet_folder, make_spectrograms,
    min_event_duration, threshold_percentile, env_smooth, color, with_labels,
    verbose, metadata, **spectrogram_kwargs)
```

audio_path	Path to the input audio file.
output_folder	Directory where the snippets and spectrograms will be saved. If None, the same directory as the input audio is used.
overlap	Overlap between consecutive snippets. Default is 0.
snippet_duration	Duration of each snippet in seconds. Default is 2 seconds.
make_spectrograms	If True, spectrograms will be generated for each snippet. Default is True.
snippet_folder	Folder with snippet to scan for events. Likely the output folder of process_audio_file.
event_snippet_folder	Directory where event snippets and spectrograms will be saved. If None, the same directory as the snippet folder will be used.
spectrogram_kwargs	Additional keyword arguments passed to the spectrogram generation function.

3.3 Extracting features

Feature_extractor() allows the user to extract all features from all acoustic events found in a folder of snippets. This is useful after a set of positive and negative snippets has been made manually in order to inspect what differentiates target and non-target sounds. See the tutorial *Prepare snippet sets*. The extracted features and their meaning can be found in Table 1&2. Note: if features must be extracted from the complete audio files (i.e., the each file should be handled as one event), set threshold percentile 0.

```
# Extract features #
st.feature_extractor(snippet_folder, output_folder, min_event_duration,
    threshold_percentile, env_smooth, log_name, df_format, **features_kwargs)
```

snippet_folder	Path to a folder with snippets.
df_format	Create a 'long' or 'wide' (default) data frame.
features_kwargs	Additional kwargs for extract_features(), including the **mfcc_kwargs**.

Table 1: Sound features included in *Stridulant* that can be used to describe sounds and define feature-based scans. 'Event' refers to a single acoustic event, which within the *Stridulant* pipeline is defined as an audio signal where the Hilbert energy envelope exceeds a user-defined percentile threshold. Because *Stridulant* is targeted mainly towards non-vocalization sounds, we incorporated several *pulse* descriptors (features six to ten). Pulses are short bursts of energy that are common in insect sounds. These descriptors quantify properties of the pulses within an event, such as their number, duration, spacing, and temporal distribution. If a number of Mel-frequency cepstral coefficients dimensions is defined, *Stridulant* will calculate statistics for each dimension (Table 2).

	Feature name	**Description**	**Unit**
1	event duration	Duration of the event	Seconds
2	event energy	Peak amplitude of the smoothed envelope	Typically unitless, unless audio is calibrated
3	attack rate	Rate at which amplitude rises in the first 20% of the event envelope	Amplitude units per sample
4	duty cycle	Fraction of event duration above an envelope threshold	Fraction (0–1)
5	dynamic range	Difference between the loudest and quietest spectral amplitudes	dB
6	avg pulse duration	Mean pulse duration	seconds
7	avg pulse interval	Mean time between pulses	seconds
8	pulse count	Number of pulses	Count per event
9	pulse density	Pulses per second	Count per second
10	pulse regularity	Rhythm consistency	Standard deviation of intervals between pulses
11	frequency range	Bandwidth containing 90% of spectral energy	Hz
12	spectral centroid mean	Dominant frequency	Hz
13	spectral roll off	Frequency at which 85% of energy falls below	Hz
14	tonal variation	Frequency stability	Standard deviation of the mean of centroid

Table 2: Statistics calculated for each Mel-frequency cepstral coefficients dimension.

	Feature name	**Description**	**Unit**
1	min.cc	Minimum value of the coefficient across frames	Coefficient units
2	max.cc	Maximum value of the coefficient across frames	Coefficient units
3	median.cc	Median value of the coefficient across frames	Coefficient units
4	mean.cc	Mean value of the coefficient across frames	Coefficient units
5	var.cc	Variance of the coefficient across frames	Coefficient units^2
6	skew.cc	Skewness (asymmetry) of the coefficient distribution	Dimensionless
7	kurt.cc	Kurtosis (tailedness/peakedness) of the coefficient distribution	Dimensionless
8	mean.d1.cc	Mean of the first derivative (velocity) of the coefficient	Coefficient units/frame
9	var.d1.cc	Variance of the first derivative (velocity) of the coefficient	Coefficient units^2/frame^2
10	mean.d2.cc	Mean of the second derivative (acceleration) of the coefficient	Coefficient units/frame^2
11	var.d2.cc	Variance of the second derivative (acceleration) of the coefficient	Coefficient units^2/frame^4

4 Feature-based scans

Based on the acoustic features of a sound, feature-based scans can be used to determine whether it fits the target sound or not. We include two predefined functions (for ant stridulation and plant ultrasonic acoustic emissions, Box 1) and work through one example of how to make your own using worm sounds. See the tutorial *feature-based scans*.

Simplified examples of feature-based scans

Stridulation scan	**UAE scan**
Event features	**Event features**
↓	↓
Does the mean spectral centroid fall between 6 and 9 kHz?	Is the spectral rolloff at least 23 kHz?
↓ **Yes**	↓ **Yes**
Are there at least 3 distinct pulses?	Is the event duration not longer than 2 ms?
↓ **Yes**	↓ **Yes**
Is the event longer than 0.1 seconds?	**UAE candidate**
↓ **Yes**	
Stridulation candidate	

Box 1: Simplified example of a feature-based scan for identifying candidate stridulation and ultrasonic acoustic emission (UAEs) events. See the case studies in the accompanying publication for more biological context.

4.1 Included functions

The included feature-based scan provide a starting off point, but they will likely have to be fine-tuned to the users specific audio files. The function *is_stridulation* allows for event coupling, meaning that two weak events will be combined together, if they are not separated more than *coupled_gap*. The standard *min_event_duration* still has to be defined in order to do the initial event scan.

```
# Ant stridulation #
snippet.is_stridulation(min_event_duration, threshold_percentile, env_smooth,
    pulse_dist, min_pulses, min_regularity, uncoupled_min_duration, max_duration,
```

```
    sustain, sp_range, enable_coupled, coupled_min_duration, coupled_gap)
# Returns True if a stridulation may be present in the snippet.

# Tree UAE #
snippet.is_uae(min_event_duration, threshold_percentile, env_smooth, pulse_dist,
    spectral_rolloff_min, min_energy, max_duration, max_energy)
# Returns True if a UAE may be present in the snippet.
```

snippet	Snippet object.
pulse_dist	Minimum distance between pulses in milliseconds. Default is 20.
min_pulses	Minimum number of individual pulses required within the event. Default is 6.
min_regularity	Minimum pulse regularity score (1 / standard deviation of intervals). Higher values indicate more consistent timing between pulses. Values >10 suggest rhythmic, organized patterns.
uncoupled_min_duration	Minimum duration for individual uncoupled events.
max_duration	Maximum event duration in seconds. Use None for no maximum. Default is 1.
sustain	Minimum duty cycle ratio (0–1) indicating what fraction of the event duration contains actual sound.
sp_range	Valid frequency range for spectral centroid in Hz (min, max).
enable_coupled	Whether to enable detection of coupled weak events. Default is True.
coupled_min_duration	Minimum duration for events in coupled detection. Allows shorter events to be considered only when looking for coupled pairs. Default is 0.22.
coupled_gap	Maximum time gap between consecutive weak events in seconds. Default is 0.6.
spectral_rolloff_min	Frequency at which 85% of energy lies below. A minimum set at 23000 ensures that at least 15% of the event's energy is in the ultrasonic range. Default is 23000.
min_energy	Minimum energy required for a UAE event.
max_energy	Maximum energy allowed for a UAE event.

4.2 User-defined feature-based scans

Below is a most basic example of a user-defined feature-based scan. For a more complicated example, see the *feature-based scans* tutorial. Ensure that throughout the process of developing a featuer-based scans, the same values for env_smooth, threshold_percentile and pulse_dist values are used, as differences in these parameters affect the outcomes.

```
# Define a function called is_test()
def is_test(snippet,
                    env_smooth = 10,
                    threshold_percentile = 20,
                    min_energy = 1,
                    pulse_dist = 20,
                    min_event_duration = .5,
                    ...
                    # Add more arguments here
                    ):

    # Detect events
    events = snippet.find_events(
        min_event_duration,
        threshold_percentile,
        env_smooth)

    if not events:
        return False

    # Extract features from events
    else:
        for event in events:
            features = snippet.extract_features(event, pulse_dist = pulse_dist)

    # Return True if any of the events match the arguments
            if (features["event_energy"] > min_energy):
                return True

        return False

# Apply function to a snippet
snippet.is_test()
```

5 Convolutional neural network

Stridulant included a basic CNN model designed for binary classification. That means that for each spectrogram, the model will be trained and ultimately decide whether a target sounds is present (True) or not (False). The architecture has three convolutional blocks, structured as follows: Input (128 × 128 × 3) → Conv2D (32, 3 × 3) → MaxPooling (2 × 2) → Conv2D (64, 3 × 3) → MaxPooling (2 × 2) → Conv2D (128, 3 × 3) → MaxPooling (2 × 2) → Flatten → Dense (512) → Dropout (0.5) → Dense (1, sigmoid).

```
from stridulant import train_model as tm, classify_spectrograms as cs

## Train model
mod, hist = tm.train_model(train_dir, batch_size, epochs, learning_rate,
    class_weights)

## Run model
cs.classify_spectrograms(spectrogram_folder, model)
```

train_dir	Path to the directory containing the training images. Should contain one folder with positive and one with negative spectrograms.
target_size	The target size to which each input image is resized (default is (128, 128)).
batch_size	The number of images per batch used during training (default is 32).
epochs	The number of epochs to train the model (default is 5).
learning_rate	The learning rate for the optimizer (default is 0.001).
class_weights	A dictionary of class weights for handling class imbalance. If None, the class weights are computed based on the class distribution.

6 Pipeline

The pipeline allows users to scan whole folders or drives for target sounds. It is designed to be able to handle interruptions. It does this by reading and writing a log file, which is also where the results are saved (Table 3). You can also use the pipeline to save all events above an energy threshold, i.e., without a method screening for target sounds.

After the feature-based scan is completed, the analysis may be followed up by training of a convolutional neural network (CNN). For example, a feature-based scan may serve as an initial filtering step, simultaneously outputting event spectrograms, which after manual refinement smoothly serve as training and testing data for CNN.

Table 3: Example of the log file created by the *Stridulant* pipeline. Snippet_start_time indicates the start time within the original audio file. In this case, snippet duration was set to 2 seconds. Event is Yes when at least one event is present in the snippet and No when the snippet is empty. Results is TRUE if at least one of the events in the snippet matches the target sound and FALSE when it does not pass the feature-based scan.

Original_audio_path	Snippet_start_time	Snippet_end_time	Event	Result
Ant_stridulation.wav	0	2	Yes	FALSE
Ant_stridulation.wav	2	4	Yes	TRUE
Ant_stridulation.wav	4	6	Yes	TRUE
Ant_stridulation.wav	6	8	No	NA
Ant_stridulation.wav	8	10	Yes	FALSE